

VC_HW2_314581003

Homework #2 - 2D-DCT

一、 2D-DCT：圖像轉換、重建與評估 PSNR

1.1 程式碼實作：

◆ def dct_2d(image):

```
def dct_2d(image):
    """公式實現 2D-DCT"""
    N, M = image.shape
    dct_matrix = np.zeros((N, M))
    c_u = get_dct_coeffs(N)
    c_v = get_dct_coeffs(M)

    # 預先計算餘弦項以加速
    cos_u = np.zeros((N, N))
    cos_v = np.zeros((M, M))
    for i in range(N):
        for u in range(N):
            cos_u[i, u] = np.cos((2 * i + 1) * u * np.pi / (2 * N))
    for j in range(M):
        for v in range(M):
            cos_v[j, v] = np.cos((2 * j + 1) * v * np.pi / (2 * M))

    print(f"\n計算 {N}x{M} 的直接 2D-DCT")
    for u in tqdm(range(N), desc="DCT - 處理 u", ncols=100):
        for v in range(M):
            # 使用 NumPy 的向量化操作加速內層迴圈
            sum_val = np.sum(image * cos_u[:, u].reshape(-1, 1) * cos_v[:, v])
            dct_matrix[u, v] = c_u[u] * c_v[v] * sum_val
    return dct_matrix
```

模擬公式中的雙重總和運算，其計算複雜度為 $O(N^4)$ 。

- **餘弦表預計算：**在進入主迴圈前，程式會預先計算所有可能會用到的 $\cos$ 值並存放在 $\cos_u$ 與 $\cos_v$ 兩個矩陣中。這個步驟是為了避免在最核心的迴圈中重複呼叫 $\text{np.cos}()$ ，以提升運算效率。
- **巢狀迴圈與向量化：**程式碼使用兩層外部迴圈遍歷輸出矩陣的每一個座標 (u, v) 。在迴圈內部，利用 NumPy 與向量化特性 $\text{np.sum}(\text{image} * \cos_u[:, u].\text{reshape}(-1, 1) * \cos_v[:, v])$ ，來取代原本的兩層內層迴圈，以加速單一係數的計算，但整體複雜度不變。

◆ idct_2d(dct_matrix):

```
def idct_2d(dct_matrix):
    """公式實現 2D-IDCT"""
    N, M = dct_matrix.shape
    image = np.zeros((N, M))
    c_u = get_dct_coeffs(N)
    c_v = get_dct_coeffs(M)

    # 預先計算餘弦項
    cos_u = np.zeros((N, N))
    cos_v = np.zeros((M, M))
    for i in range(N):
        for u in range(N):
            cos_u[i, u] = np.cos((2 * i + 1) * u * np.pi / (2 * N))
    for j in range(M):
        for v in range(M):
            cos_v[j, v] = np.cos((2 * j + 1) * v * np.pi / (2 * M))

    print("\n執行 2D-IDCT 重建影像...")
    for i in tqdm(range(N), desc="IDCT - 重建 i", ncols=100):
        for j in range(M):
            # 準備係數矩陣
            coeffs_matrix = c_u.reshape(-1, 1) * c_v * dct_matrix
            # 向量化計算
            sum_val = np.sum(coeffs_matrix * cos_u[i, :].reshape(-1, 1) * cos_v[j, :])
            image[i, j] = sum_val
    return image
```

此函式為反向轉換的實現，其結構與 dct_2d 高度相似，同樣透過預計算餘弦表與向量化的方式，將頻域的 DCT 係數轉換回空間域的像素值。

◆ calculate_psnr(img1, img2):

```
def calculate_psnr(img1, img2):
    """Evaluate the PSNR"""
    img1 = img1.astype(np.float64)
    img2 = img2.astype(np.float64)
    mse = np.mean((img1 - img2) ** 2)
    if mse == 0:
        return float('inf')
    psnr = 20 * math.log10(255.0 / math.sqrt(mse))
    return psnr
```

首先計算原始圖像與重建圖像之間的 MSE，然後將其代入 PSNR 公式 $20\log_{10}(255/\text{MSE})$ 轉換為以分貝(dB)為單位的對數值。

1.2 執行結果

將 512x512 的 lena.png 灰階影像輸入上述函式進行運算，得到以下結果。



分析：如中間圖所示，影像能量成功集中在左上角的低頻區域，符合 DCT。

● PSNR 評估：

重建圖像（右圖）與原始圖像（左圖）之間的 PSNR 計算結果為：

■ PSNR 結果： 274.5373 dB

分析：極高的 PSNR 值表明重建圖像與原始圖像的誤差極小，幾乎達到無損（微小差異來自電腦浮點數運算的精度限制）。

二、 Two 1D-DCT：快速演算法與效能比較

2.1 程式碼實作

◆ `dct_1d(vector):`

```
def dct_1d(vector):  
    """實現 1D-DCT"""  
    N = len(vector)  
    result = np.zeros(N)  
    c = get_dct_coeffs(N)  
    for k in range(N):  
        n = np.arange(N)  
        cos_terms = np.cos((2 * n + 1) * k * np.pi / (2 * N))  
        sum_val = np.sum(vector * cos_terms)  
        result[k] = c[k] * sum_val  
    return result
```

這個函式是快速演算法的核心構建模組。它接收一個一維陣列（即圖像的一行或一列），並對其進行 1D-DCT 運算後，返回一個包含頻率係數的一維陣列。其內部同樣透過 NumPy 向量化來加速總和運算。

◆ `two_1d_dct(image):`

```
def two_1d_dct(image):  
    """使用Two 1D-DCT 實現 2D-DCT"""  
    N, M = image.shape  
    intermediate = np.zeros((N, M))  
  
    # 對每一列進行 1D-DCT  
    for i in tqdm(range(N), desc="Two 1D-DCT (each row)", ncols=100):  
        intermediate[i, :] = dct_1d(image[i, :])  
  
    final_dct = np.zeros((N, M))  
    # 對每一行進行 1D-DCT  
    for j in tqdm(range(M), desc="Two 1D-DCT (each column)", ncols=100):  
        final_dct[:, j] = dct_1d(intermediate[:, j])  
  
    return final_dct
```

此函式實現了完整的快速 2D-DCT，其計算複雜度為 $O(N^3)$ 。

- 第一階段 (逐列轉換)：函式首先初始化一個中間矩陣 `intermediate`。接著，它遍歷原始圖像的每一列 (row)，將每一列傳遞給 `dct_1d` 函式進行運算，並將返回的一維係數陣列存回 `intermediate` 矩陣的對應位置。
- 第二階段 (逐行轉換)：在所有列都轉換完畢後，函式接著遍歷 `intermediate` 矩陣的每一行 (column)，同樣將每一行傳遞給 `dct_1d` 函式進行運算，最終得到完整的 2D-DCT 結果。

這個兩階段的處理流程，完美地利用了 2D-DCT 的可分離性，避免了原本 2D-DCT 高複雜度的四層迴圈。

2.2 執行時間比較

對 512x512 的灰階影像分別執行 dct_2d 與 two_1d_dct，得到執行時間如下。

- **2D-DCT 執行時間：308.8448 秒**

- **Two 1D-DCT 執行時間：7.3330 秒**

分析：實驗數據明確顯示，快速演算法的執行效率約為直接公式法的 42 倍 ($308.8448 / 7.3330 \approx 42$)。此結果強而有力地證明了，利用演算法理論進行優化，在處理實際問題時能帶來巨大的效能提升。

三、 結論

本次作業成功地從零開始實現了 DCT/IDCT 的直接與快速兩種演算法，並透過程式碼解析與實驗數據驗證了其原理與效能。我們的程式碼執行結果表明：

- 直接法與快速法均能正確實現 DCT 轉換，並透過 IDCT 進行高精度重建 (PSNR > 270 dB)。
- 快速演算法 Two 1D-DCT 在效能上具有壓倒性優勢，其執行速度比直接法快了約 42 倍。

附：結果輸出

```
===== Results =====
分析影像尺寸：512x512
2D-DCT 執行時間：308.8448 秒
重建圖像的 PSNR：274.5373 dB
Two 1D-DCT 執行時間：7.3330 秒
=====
```